

UA-DETI - Arquitetura de Software

Architecture Report

Architectural Evolution of nopCommerce
Cenário C — Omnichannel Commerce Core

Curso: AS - Arquitetura de Software
Docentes: Cláudio Teixeira
Alunos: 113655: Abel Teixeira
131224: Francisco Costa
131690: Guilherme Pinho
112901: Tiago Albuquerque
Data: Junho 2026

Índice

1. Contexto.....	4
1.1 Introdução e contextualização.....	4
2. Cenários.....	5
2.1 Os 3 cenários.....	5
2.2 Cenário escolhido.....	5
3. Análise ao estado atual.....	6
3.1 Estrutura arquitetural.....	6
3.2 Funcionalidades relevantes.....	6
3.3 Pontos fortes da arquitetura atual.....	6
3.4 Limitações face ao cenário escolhido.....	7
3.5 Pontos de evolução.....	7
3.6 Modelação do domínio.....	7
3.7 Bounded contexts.....	8
3.8 Responsabilidades (Ownership).....	8
3.9 Conclusão.....	8
4. Motivações — de negócio e arquiteturas.....	9
4.1 Drivers na lógica de negócio.....	9
4.2 Drivers de arquitetura.....	9
5. Quality Attribute Scenarios.....	10
6. Framework — Desenho arquitetural.....	12
6.1 ADD (Attribute-Driven Design).....	12
6.2 Aplicação do ADD à implementação final.....	12
7. Arquitetura alvo.....	13
7.1 Componentes e serviços principais.....	13
7.2 Data ownership.....	14
7.3 Interações síncronas e assíncronas.....	14
7.3.1 Interações síncronas.....	14
7.3.2 Interações assíncronas.....	14
7.4 Cross-cutting concerns.....	15
7.5 Conclusão.....	15
8. Arquitetura implementada — Trabalho realizado.....	16
8.1 Alterações realizadas.....	16
8.2 Fluxo implementado — checkout com publicação assíncrona.....	18
8.3 Estrutura do OrderCreatedEvent.....	18
8.4 Configuração RabbitMQ.....	19
8.5 Order Integration Service.....	19
8.6 MockStockService — simulação do WMS.....	19
8.7 MockShippingService — simulação do Shipping Service.....	20
8.8 Integration Status UI.....	20
8.9 Cenário de operação degradada e respetiva recuperação.....	21
9. ADRs — Architectural Decision Records.....	22

10. Plano de risco e estratégias de mitigação.....	25
10.1 Riscos arquiteturais.....	25
10.2 Estratégias de mitigação adotadas.....	26
10.3 Estratégias de mitigação futuras.....	26
10.4 Conclusão.....	26
11. Evolution Roadmap.....	27
11.1 Fases de implementação.....	27
11.2 Ordem de evolução.....	28
11.3 Coexistências durante a transição.....	28
11.4 Conclusão.....	28
12. Feasibility Spike.....	29
12.1 Objetivo.....	29
12.2 Descrição e resultados — fluxo assíncrono completo.....	29
12.3 12.3 Runtime challenge demonstrável — Order Integration Service indisponível....	30
12.4 Conclusão.....	30
13. Runtime challenge e demonstração.....	31
13.1 Operação normal — checkout com processamento externo assíncrono.....	31
13.2 Operação com o Order Integration Service indisponível.....	31
13.3 Recuperação — consumidor volta a estar disponível.....	32
13.4 Visibilidade do estado externo da encomenda.....	32
14. Evidence Pack.....	33
14.1 Evidência de implementação.....	33
15. Rastreabilidade — drivers, QA, ADRs e implementação.....	34
16. Conclusão.....	35

1. Contexto

1.1 Introdução e contextualização

Este tipo de sistemas, como o nopCommerce, têm vindo a evoluir significativamente, deixando de funcionar como plataformas isoladas e passando a integrar ecossistemas mais amplos e interligados. Em contextos empresariais modernos, é comum que diferentes funcionalidades — como gestão de stock, processamento de encomendas, faturação e questões logísticas — estejam distribuídas por múltiplos sistemas especializados.

Neste contexto, a plataforma open-source nopCommerce foi utilizada como base para o desenvolvimento deste projeto. Esta plataforma representa uma solução de e-commerce com uma arquitetura estruturada em camadas, que disponibiliza funcionalidades essenciais como a gestão do catálogo, de compras e de encomendas, assim como a gestão dos respetivos clientes.

O presente documento constitui o **Architecture Report (Parte 2)** do trabalho final de grupo. Integra a análise arquitetural e as decisões documentadas na Parte 1 com a implementação realizada e as evidências de comportamento demonstrável. O objetivo do trabalho consiste em analisar a arquitetura atual da plataforma e explorar e implementar caminhos de evolução que respondam a novos desafios de integração, escalabilidade e adaptação a diferentes contextos de negócio.

2. Cenários

2.1 Os 3 cenários

No âmbito deste projeto foram apresentados três cenários distintos, representativos de diferentes perspectivas arquitetônicas no contexto da evolução da plataforma nopCommerce. Cada um introduz desafios específicos ao nível da organização do sistema, da integração com outros componentes e da gestão de requisitos não funcionais.

- **Cenário A — Federated Commerce After Acquisitions:** aborda crescimento empresarial através de aquisições, onde múltiplas unidades de negócio coexistem sob uma mesma estrutura organizacional, exigindo equilíbrio entre base digital partilhada e autonomia local.
- **Cenário B — Regulated / Controlled Commerce:** retrata o condicionamento do comércio por requisitos de regulamentação, validação e auditoria, com suporte a políticas dinâmicas, aprovações e gestão de evidência documental.
- **Cenário C — Omnichannel Commerce Core:** foca-se na evolução da plataforma para núcleo central de um ecossistema mais alargado, onde múltiplos sistemas (ERP, WMS, shipping, POS) colaboram para suportar o ciclo completo de uma encomenda.

2.2 Cenário escolhido

Optámos por seleccionar o **Cenário C — Omnichannel Commerce Core**. O objetivo passa por evoluir a plataforma nopCommerce de uma plataforma e-commerce para um núcleo central de coordenação de operações no âmbito do comércio.

Atualmente, o nopCommerce funciona principalmente como uma loja online autónoma, sendo responsável por gerir catálogo, carrinho, encomendas e o respetivo estado. No entanto, em contextos empresariais reais, estas responsabilidades encontram-se frequentemente distribuídas por múltiplos sistemas especializados, tais como ERP, sistemas de gestão de armazém (WMS), serviços de shipping e plataformas de ponto de venda (POS).

O principal objetivo é tratar de uma evolução do atual nopCommerce para um **commerce coordination core**, capaz de integrar e coordenar múltiplos sistemas externos, mantendo-se funcional e consistente mesmo perante atrasos, falhas ou inconsistências nesses sistemas.

3. Análise ao estado atual

A plataforma nopCommerce apresenta uma arquitetura baseada numa abordagem em camadas. Esta estrutura permite uma separação lógica entre diferentes responsabilidades do sistema, nomeadamente ao nível da apresentação, lógica de negócio e acesso a dados.

3.1 Estrutura arquitetural

A arquitetura do nopCommerce pode ser caracterizada pelas seguintes camadas principais:

- **Presentation Layer:** Responsável pela interação com o utilizador, incluindo a interface web e os controladores que tratam os pedidos HTTP.
- **Application / Services Layer:** Contém a lógica de negócio da aplicação, incluindo serviços responsáveis pela gestão de produtos, encomendas, clientes e outros processos fundamentais.
- **Domain Layer:** Representa as entidades e modelos de negócio centrais do sistema.
- **Data Access Layer:** Responsável pela persistência de dados, garantindo a comunicação com a base de dados.

3.2 Funcionalidades relevantes

No contexto deste projeto, destacam-se as seguintes funcionalidades já suportadas pela plataforma:

- Gestão de catálogo de produtos
- Carrinho de compras e processo de checkout
- Criação e gestão de encomendas
- Gestão de clientes
- Consulta do estado das encomendas pelo utilizador

Estas funcionalidades permitem ao nopCommerce operar todo um ciclo de compra, desde a seleção de produtos até à finalização da respetiva encomenda.

3.3 Pontos fortes da arquitetura atual

- **Centralização do fluxo de negócio:** Todo o ciclo de encomenda é gerido internamente, garantindo controlo total sobre o estado da aplicação.
- **Coerência e simplicidade:** A estrutura baseada num sistema central facilita o desenvolvimento e a compreensão do sistema.
- **Modularidade interna:** A separação por camadas e serviços permite alguma extensibilidade e organização do código.
- **Maturidade da plataforma:** Sendo uma solução amplamente utilizada, apresenta estabilidade e um conjunto robusto de funcionalidades.

3.4 Limitações face ao cenário escolhido

- **Dependência de estado interno:** O sistema assume controlo direto sobre entidades como stock e estado das encomendas, não estando preparado para lidar com múltiplas fontes externas.
- **Integração limitada com sistemas externos:** A arquitetura não está orientada para integração robusta com sistemas como ERP, WMS ou serviços de shipping.
- **Predominância de fluxos síncronos:** A maior parte das operações ocorre de forma síncrona, dificultando a gestão de atrasos ou falhas externas.
- **Acoplamento ao sistema central:** A lógica de negócio encontra-se fortemente concentrada, dificultando a extração de componentes independentes.

3.5 Pontos de evolução

- **Introdução de integração assíncrona:** Necessidade de suportar comunicação baseada em eventos entre o nopCommerce e sistemas externos.
- **Separação de responsabilidades:** Possibilidade de extrair componentes responsáveis pela integração com sistemas externos, reduzindo o acoplamento ao núcleo da aplicação.
- **Gestão de consistência eventual:** Adaptação do sistema para lidar com estados temporariamente inconsistentes entre diferentes sistemas.
- **Introdução de mecanismos de fiabilidade:** Implementação de estratégias como retries, idempotência e persistência de eventos.
- **Melhoria da observabilidade:** Necessidade de monitorizar fluxos distribuídos e identificar falhas em integrações externas.

3.6 Modelação do domínio

Tendo em conta o cenário selecionado e as limitações identificadas na arquitetura atual, torna-se relevante analisar a organização do domínio do sistema e definir fronteiras claras de responsabilidade entre os diferentes componentes.

Podem ser identificados os seguintes subdomínios principais no contexto Omnichannel:

- **Experiência de Comércio:** Responsável pela interação com o utilizador, incluindo navegação no catálogo, carrinho de compras, checkout e consulta de encomendas.
- **Gestão de Encomendas:** Responsável pela criação e gestão do estado lógico das encomendas dentro do sistema.
- **Fulfillment / Gestão de Stock (externo):** Responsável pela gestão de stock real, reserva de produtos e preparação de encomendas, tipicamente suportado por sistemas WMS.
- **Entrega / Shipping (externo):** Responsável pelo processamento de envios e atualização do estado de entrega das encomendas.

3.7 Bounded contexts

Com base nos subdomínios identificados, foram definidos os seguintes bounded contexts:

Commerce Experience Context	Responsável pela experiência do utilizador e interação com o sistema. Localiza-se no nopCommerce existente.
Order Management Context	Responsável pela criação e gestão das encomendas, bem como pela emissão de eventos relevantes para integração externa. Localiza-se no nopCommerce existente.
Order Integration Context	Responsável pela coordenação entre o nopCommerce e os sistemas externos, incluindo comunicação, retries e gestão de inconsistência.
Warehouse Context — WMS	Responsável pela gestão de stock e preparação de encomendas.
Shipping Context	Responsável pelo processamento de envios e atualização de estados de entrega.

Como já foi referido, nesta implementação, os serviços Warehouse e Shipping são representados por mocks integrados no Order Integration Service. Conseguimos assim demonstrar a comunicação assíncrona, possíveis falhas e respetiva recuperação e a visibilidade do estado externo sem exigir a implementação completa de sistemas WMS e Shipping reais.

3.8 Responsabilidades (Ownership)

A definição clara de responsabilidades entre contextos é fundamental para garantir desacoplamento e evolução independente do sistema. O nopCommerce mantém responsabilidade sobre a experiência do utilizador, a criação de encomendas e o estado visível ao cliente. O Order Integration Context coordena os fluxos entre sistemas e gere retries e consistência eventual. O WMS e o Shipping Context possuem os dados de stock, preparação e entrega, respetivamente.

3.9 Conclusão

O nopCommerce apresenta uma arquitetura sólida e adequada para o seu propósito original como plataforma de e-commerce autónoma. No entanto, para responder às exigências de um cenário omnichannel, torna-se necessário evoluir a arquitetura no sentido de suportar integração com múltiplos sistemas externos, promover desacoplamento e introduzir mecanismos de resiliência e consistência eventual. Esta análise permitiu identificar os principais pontos de intervenção arquitetural que serviram de base para a definição da arquitetura alvo.

4. Motivações — de negócio e arquiteturais

É fundamental identificar os principais fatores que motivam a evolução da arquitetura do sistema, tendo em conta o cenário escolhido. Estes fatores dividem-se em drivers de negócio e drivers arquiteturais.

4.1 Drivers na lógica de negócio

- **Integração com múltiplos sistemas externos:** A plataforma deve ser capaz de integrar sistemas como ERP, WMS e serviços de shipping, permitindo suportar operações distribuídas.
- **Experiência unificada para o cliente:** O utilizador deve ter uma visão consistente do estado das suas encomendas, independentemente dos sistemas envolvidos no seu processamento.
- **Continuidade de operação:** O sistema deve manter-se funcional mesmo quando existem falhas ou atrasos em sistemas externos.
- **Visibilidade do ciclo de encomenda:** Deve ser possível acompanhar o estado das encomendas ao longo de todo o processo, incluindo etapas externas ao nopCommerce.
- **Capacidade de evolução:** A arquitetura deve permitir a integração futura de novos sistemas ou canais sem impacto significativo no núcleo do sistema.

4.2 Drivers de arquitetura

- **Resiliência:** O sistema deve ser capaz de lidar com falhas e indisponibilidade de sistemas externos sem comprometer a sua funcionalidade.
- **Consistência eventual:** Deve ser aceite que o estado do sistema pode ser temporariamente inconsistente, sendo garantida a sua correção ao longo do tempo.
- **Integração assíncrona:** A comunicação entre o nopCommerce e sistemas externos deve ser baseada em mecanismos assíncronos, reduzindo o acoplamento.
- **Desacoplamento entre componentes:** Os diferentes contextos do sistema devem ter responsabilidades bem definidas e baixo acoplamento entre si.
- **Observabilidade:** O sistema deve permitir monitorizar e analisar o comportamento das integrações e identificar falhas.
- **Modularidade e extensibilidade:** A arquitetura deve permitir a evolução do sistema de forma incremental, facilitando a introdução de novos componentes.

5. Quality Attribute Scenarios

Com base nos fatores de negócio e de arquitetura identificados, definem-se os seguintes cenários de qualidade. Cada cenário indica o seu estado de validação: Validado (demonstrável com os artefactos implementados), Parcialmente validado (decisão tomada e implementada em parte) ou A validar (depende do serviço externo mock).

QA1 — Resiliência a falhas externas	
Fonte	Sistema de gestão de armazém (WMS).
Estímulo	Indisponibilidade do sistema durante o processamento de uma encomenda.
Ambiente	Sistema em funcionamento normal, com encomendas a serem processadas.
Resposta	A encomenda é registada no nopCommerce, sendo colocada num estado intermédio. O sistema reencaminha automaticamente a operação para um retry assíncrono, sem bloquear a experiência do utilizador.
Métrica	Nenhuma perda de encomendas; recuperação automática após restabelecimento do sistema externo.

Conseguimos demonstrar este cenário ao pararmos o container relativo ao Order Integration Service. Com isto, a mensagem permanece no RabbitMQ e é processada quando o serviço volta a estar disponível.

QA2 — Consistência eventual (Parcialmente validado)	
Fonte	Sistema de shipping.
Estímulo	Atraso na atualização do estado de envio de uma encomenda.
Ambiente	Após criação da encomenda, com integração ativa com sistemas externos.
Resposta	O nopCommerce apresenta um estado intermédio ao utilizador, sendo posteriormente atualizado quando a informação chega do sistema externo.
Métrica	Estado final consistente em tempo aceitável (ex.: alguns minutos), sem inconsistências permanentes.

Este cenário é parcialmente validado porque o estado externo da encomenda fica visível na página criada do Order Integration Service, ainda que não seja definido e partilhado diretamente na UI original do nopCommerce.

QA3 — Desacoplamento e integração

Fonte	Equipa de desenvolvimento.
Estímulo	Integração com um novo sistema externo.
Ambiente	Sistema em produção.
Resposta	A integração é realizada através de um componente dedicado, sem necessidade de alterar significativamente o núcleo do nopCommerce.
Métrica	Alterações localizadas e tempo de integração reduzido, sem alterações significativas ao núcleo do sistema.

Este cenário é validado porque, agora, o nopCommerce apenas publica os eventos, não conhecendo diretamente WMS nem Shipping.

QA4 — Observabilidade e diagnóstico

Fonte	Operador técnico.
Estímulo	Falha na comunicação com um sistema externo.
Ambiente	Sistema em produção.
Resposta	O sistema disponibiliza logs que permitem acompanhar o fluxo da operação entre o nopCommerce e sistemas externos, identificando pontos de falha ou latência.
Métrica	Capacidade de identificar a causa da falha em tempo reduzido.

Parcialmente validado: logs estruturados no publisher e no checkout estão implementados (alteração implementada). Interface de gestão RabbitMQ disponível. Tracing distribuído entre serviços fica para fase seguinte.

QA5 — Continuidade de operação

Fonte	Utilizador final.
Estímulo	Atraso ou falha num sistema externo durante o processo de compra.
Ambiente	Durante a utilização normal da plataforma.
Resposta	O sistema mantém-se funcional, permitindo ao utilizador concluir a encomenda, mesmo que algumas operações externas sejam processadas posteriormente.
Métrica	Operação contínua sem interrupção da experiência do utilizador.

Assim, os cenários de qualidade definidos refletem os principais desafios associados à evolução do nopCommerce para este contexto Omnichannel, nomeadamente ao nível da resiliência, integração e consistência.

6. Framework — Desenho arquitetural

6.1 ADD (Attribute-Driven Design)

Para apoiar a evolução arquitetural proposta, selecionámos o método ADD (Attribute-Driven Design). Esta escolha justifica-se pelo facto de o projeto partir de um conjunto claro de atributos de qualidade e de necessidades arquitetónicas, nomeadamente resiliência, consistência eventual, integração assíncrona e observabilidade. O ADD revela-se adequado por orientar o desenho da arquitetura a partir desses fatores, permitindo que as decisões estruturais sejam diretamente influenciadas pelos cenários de qualidade previamente definidos.

Além disso, este método ajusta-se bem ao objetivo central: não se pretende redesenhar integralmente a plataforma nopCommerce, mas sim definir uma evolução arquitetural focada, justificando o que deve permanecer no núcleo da aplicação, o que deve ser separado e de que forma os diferentes componentes devem interagir.

6.2 Aplicação do ADD à implementação final

A aplicação desta framework começou pela identificação das principais questões arquiteturais subjacentes ao Cenário: resiliência a falhas externas, continuidade de operação durante o checkout, integração assíncrona, desacoplamento entre o nopCommerce e sistemas externos, observabilidade e consistência eventual.

Com base nessas questões, foram priorizados os cinco cenários referidos no capítulo anterior. Estes cenários influenciaram diretamente as decisões arquiteturais implementadas:

- **QA1 — Resiliência a falhas externas:** levou à definição e introdução do RabbitMQ como message broker. Esta decisão permite preservar eventos mesmo quando o consumidor externo não está disponível.
- **QA2 — Consistência eventual:** levou à criação de um estado de integração externo à encomenda do nopCommerce, mantido pelo Order Integration Service e exposto através da Integration Status UI. Assim, o estado externo da encomenda pode ser atualizado de forma assíncrona após o checkout.
- **QA3 — Desacoplamento e integração:** visível na introdução do Order Integration Service como um serviço independente, responsável por consumir eventos de encomenda e coordenar a comunicação com sistemas externos, sem que o nopCommerce conheça diretamente o WMS ou o Shipping Service.
- **QA4 — Observabilidade e diagnóstico.**
- **QA5 — Continuidade de operação:** separação entre o fluxo síncrono de checkout e o processamento externo. Com isto, o checkout termina no nopCommerce sem depender diretamente da disponibilidade do WMS, Shipping Service ou Order Integration Service.

Desta forma, o ADD permitiu transformar os requisitos de qualidade em decisões arquiteturais concretas e demonstráveis.

7. Arquitetura alvo

Novamente olhando para o cenário selecionado, os fatores identificados e os cenários de qualidade definidos anteriormente, a arquitetura alvo proposta procura evoluir o nopCommerce de uma plataforma centrada apenas na experiência de compra online para um núcleo de coordenação de operações de comércio num contexto omnichannel.

O objetivo não passa por substituir integralmente a arquitetura atual, mas sim por introduzir novos componentes e mecanismos que permitam integrar sistemas externos de forma desacoplada, resiliente e observável. A visão alvo é apresentada na totalidade; os elementos ainda não implementados são identificados explicitamente.

7.1 Componentes e serviços principais

- **nopCommerce:** Mantém-se como sistema central da solução, responsável pela experiência do utilizador, catálogo, carrinho, checkout, criação de encomendas e consulta do respetivo estado.
- **Transactional Outbox:** Componente responsável por persistir eventos de encomenda na mesma fronteira transacional da criação da encomenda e por encaminhá-los posteriormente para o RabbitMQ de forma assíncrona.
- **Message Broker — RabbitMQ:** Componente responsável pela comunicação assíncrona entre o nopCommerce e os serviços de integração. Configurado com exchange de tópicos, filas dedicadas e DLQ.
- **Order Integration Service:** Componente dedicado à integração entre o nopCommerce e os sistemas externos. Consome eventos do broker e coordena comunicação com WMS e Shipping Service, gerindo retries, idempotência e tratamento de falhas.
- **Warehouse Management System — WMS [implementado com MockStockService]:** Sistema externo responsável pela gestão de stock real, reserva de produtos e preparação de encomendas.
- **Shipping Service [implementado com MockShippingService]:** Sistema externo responsável pelo processamento de envios e atualização do estado de entrega.
- **Integration Status UI:** Página exposta pelo Order Integration Service que permite consultar o estado externo de processamento das encomendas. Aqui podemos ter acesso a informações por encomenda, como por exemplo, o estado de integração, a reserva de stock, a criação de envio, o tracking code e os respetivos timestamps de criação/atualização.

7.2 Data ownership

A arquitetura proposta define fronteiras claras de responsabilidade sobre os dados de cada contexto. Desta forma, evita-se uma partilha direta da base de dados entre os componentes, garantindo um maior desacoplamento e uma evolução independente.

nopCommerce	Catálogo apresentado ao utilizador, carrinho de compras, encomendas, estado visível ao cliente.
Order Integration Service	Eventos processados, estado de retries, informação de correlação, registos de idempotência.
WMS	Stock real, reserva de produtos, preparação de encomendas.
Shipping Service	Criação de envios, tracking, estado de entrega.

7.3 Interações síncronas e assíncronas

7.3.1 Interações síncronas

As interações síncronas permanecem associadas à experiência direta do utilizador dentro do nopCommerce, incluindo navegação no catálogo, operações sobre o carrinho, checkout, submissão inicial da encomenda e consulta do estado. Estas operações exigem resposta imediata ao utilizador e mantêm-se concentradas no sistema central.

7.3.2 Interações assíncronas

As interações com sistemas externos passam a ser feitas de forma predominantemente assíncrona, nomeadamente:

- emissão de eventos de criação de encomenda;
- comunicação entre nopCommerce e Order Integration Service via message broker;
- atualização de stock e fulfillment através do WMS;
- atualização do estado de envio através do Shipping Service.

Este modelo permite que o sistema continue operacional mesmo perante atrasos, falhas temporárias ou indisponibilidade de dependências externas, suportando consistência eventual entre os diferentes contextos.

7.4 Cross-cutting concerns

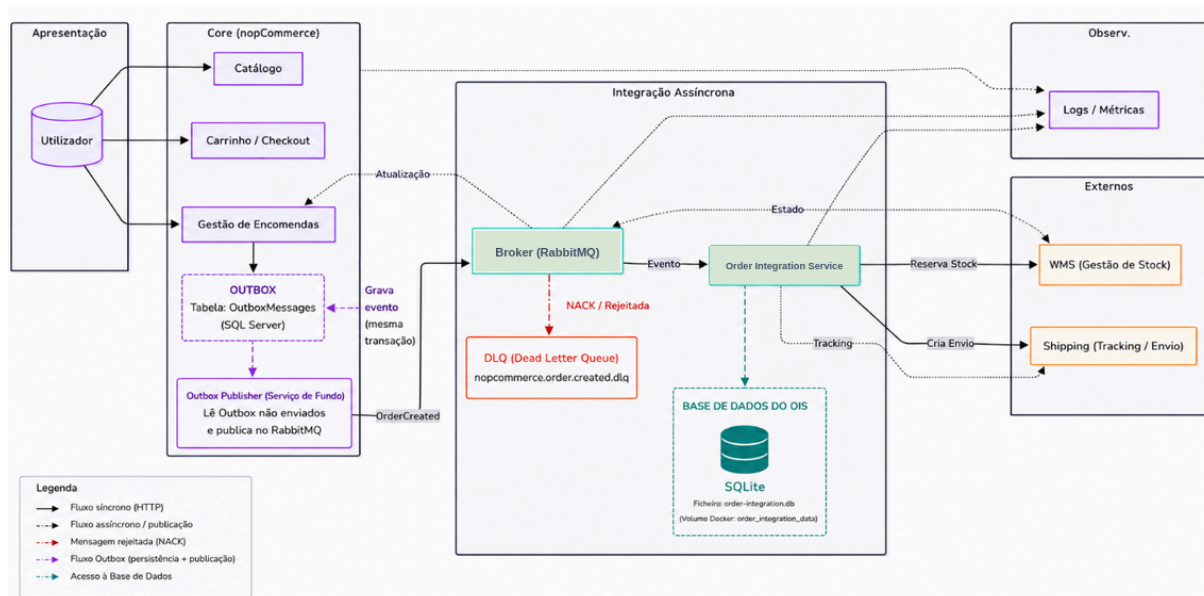
- **Resiliência:** A arquitetura tolera falhas e atrasos em sistemas externos, através de mecanismos como retries e processamento assíncrono. Implementado: try/catch no checkout, Publisher Confirms, DLQ, outbox.
- **Consistência eventual:** Aceita-se que o estado visível no nopCommerce possa não refletir imediatamente o estado final dos sistemas externos, sendo garantida a sua reconciliação ao longo do tempo.
- **Desacoplamento:** A introdução do RabbitMQ como message broker permite reduzir o acoplamento direto entre o nopCommerce e os sistemas externos.
- **Observabilidade:** Logs estruturados no publisher e no checkout; interface de gestão RabbitMQ para monitorização de filas e DLQ. Tracing distribuído planeado para fase seguinte.
- **Extensibilidade:** A arquitetura deve permitir a integração futura de novos sistemas externos sem necessidade de alterações profundas no núcleo da aplicação.
- **Fiabilidade:** O padrão Outbox elimina a janela de perda entre checkout e broker; Publisher Confirms e DLQ complementam a entrega ao broker e o tratamento de falhas de consumo

7.5 Conclusão

Esta arquitetura proposta mantém o nopCommerce como sistema central da experiência de comércio, mas introduz novos componentes e mecanismos que lhe permitem atuar como núcleo de coordenação num ecossistema distribuído. A infraestrutura assíncrona, o padrão Outbox, o Order Integration Service e a integração com WMS/Shipping simulados foram implementados. A substituição dos mocks por sistemas externos reais fica como evolução futura.

8. Arquitetura implementada — Trabalho realizado

As sucessivas implementações decorreram partindo de uma versão base do nopCommerce previamente analisada. As alterações realizadas foram ao encontro da apresentação inicial, que representava a ideia principal que sustentou as mesmas alterações nesta segunda fase.



8.1 Alterações realizadas

Área	Descrição	Impacto Arquitetural
Evento	Criação do <i>OrderCreatedEvent</i>	Estabelece o contrato do evento de integração — fronteira entre <i>Order Management Context</i> e <i>Order Integration Context</i> . Introduce EventId único e tipo fixo 'OrderCreated' para tracing e idempotência.
Persistência transacional	Introdução do padrão Outbox	Após a criação da encomenda, o nopCommerce persiste o evento 'OrderCreated' na mesma operação transacional. Um relay assíncrono publica posteriormente o evento no <i>RabbitMQ</i> , eliminando a janela de perda entre checkout e broker.
Publicação de eventos	Relay de publicação assíncrona	O evento guardado no outbox é serializado e publicado no <i>RabbitMQ</i> de forma desacoplada do fluxo principal de checkout. O processamento externo passa a depender do relay e não da execução síncrona do checkout.
Broker	Integração RabbitMQ (configuração inicial)	Introduce o Message Broker na arquitetura. Cria o publisher, configura o docker-compose e define exchanges, filas e bindings no <code>definitions.json</code> . Concretiza ADR 1 (comunicação assíncrona) e ADR 4 (RabbitMQ).
Order Integration Service	Criação de serviço independente	Serviço externo ao nopCommerce responsável por consumir eventos de encomenda, coordenar o processamento externo e gerir o acknowledgement das mensagens.

WMS mock	Implementação do 'MockStockService'	Simula um sistema externo de gestão de stock, responsável pela reserva de produtos após a criação da encomenda.
Shipping mock	Implementação do 'MockShippingService'	Simula um sistema externo de shipping, responsável pela criação do envio e geração de um tracking code.
Persistência do OIS	Base de dados SQLite própria do Order Integration Service	O OIS persiste o estado externo das encomendas realizadas numa base de dados própria, guardada no volume Docker <i>order_integration_data</i> . Isto evita a perda do histórico em caso de ocorrência de alguma falha que implique com o correto funcionamento do OIS.
Visibilidade de estado	Criação da Integration Status UI	Página que apresenta o estado externo da encomenda com as seguintes informações: integração, stock, shipping, tracking e timestamps.
Fiabilidade	Conclusão da configuração RabbitMQ e do Outbox	Consolida a resiliência do fluxo de integração: Outbox transacional, relay assíncrono, timeouts, Publisher Confirms (WaitForConfirms), flag mandatory, DLQ e registo no DI. Fluxo de checkout protegido por try/catch como salvaguarda operacional — valida QA5 (continuidade de operação).
Observabilidade	Logs, métricas e dashboards	A solução inclui logs no nopCommerce e no OIS, métricas do Prometheus, dashboards no Grafana e logs em Loki, permitindo acompanhar o fluxo de encomendas e diagnosticar falhas.
Operação e readiness	Healthcheck do RabbitMQ	O 'docker-compose' foi atualizado para garantir que o nopCommerce e o Order Integration Service arrancam apenas depois do RabbitMQ estar saudável.

A arquitetura implementada mantém o nopCommerce como sistema central da experiência de compra, responsável pelo catálogo, carrinho, checkout e criação de encomendas. Após o checkout, é criado um evento *OrderCreated*, que é guardado no Outbox do nopCommerce na mesma transação da encomenda. Posteriormente, um relay assíncrono lê os eventos pendentes e publica-os no RabbitMQ.

Esta abordagem reduz o risco de perda de eventos, uma vez que, se o RabbitMQ estiver temporariamente indisponível, o evento permanece no Outbox até poder ser reenviado. O RabbitMQ atua como broker de mensagens, desacoplando o nopCommerce do processamento externo.

O Order Integration Service consome os eventos do RabbitMQ e coordena o processamento da encomenda. Nesta implementação, o WMS e o Shipping Service são representados por mocks: o *MockStockService* simula a reserva de stock e o *MockShippingService* simula a criação do envio e a geração de um tracking code.

Após o processamento, o OIS guarda o estado da integração numa base de dados SQLite própria, persistida num volume Docker. Assim, o histórico dos checkouts processados mantém-se mesmo após reinício dos containers.

A solução inclui ainda observabilidade através de logs, RabbitMQ Management UI, métricas Prometheus, dashboards Grafana, logs em Loki e uma Integration Status UI. No conjunto, a arquitetura demonstra comunicação assíncrona, desacoplamento, continuidade de operação, recuperação e visibilidade do estado externo da encomenda.

8.2 Fluxo implementado — checkout com publicação assíncrona

O fluxo implementado funciona da seguinte forma:

- O utilizador completa o checkout no nopCommerce.
- O nopCommerce cria a encomenda na sua base de dados.
- Após a criação da encomenda, é criado um **OrderCreatedEvent**.
- O evento é persistido no outbox transacional do nopCommerce.
- Um relay assíncrono lê os eventos pendentes do outbox e publica-os no exchange **nopcommerce.order.events**, com a routing key **order.created**
- O RabbitMQ encaminha a mensagem para a fila **nopcommerce.order.created**.
- O Order Integration Service consome a mensagem da fila.
- O serviço atualiza o estado interno da encomenda para **Received**.
- O **MockStockService** simula a reserva de stock e o estado passa para **Reserved**.
- O **MockShippingService** simula a criação do envio e gera um tracking code.
- O estado da encomenda passa para **Completed**.
- O Order Integration Service envia ACK ao RabbitMQ.
- A mensagem é removida da fila.
- A Integration Status UI apresenta a encomenda com o estado final de integração.

Este fluxo permite que o checkout continue a ser uma operação controlada pelo nopCommerce, enquanto a durabilidade do evento é assegurada pelo outbox e o processamento externo é realizado de forma assíncrona e desacoplada.

8.3 Estrutura do OrderCreatedEvent

EventId (Guid)	Identificador único do evento para idempotência e tracing.
EventType (string)	Tipo fixo <i>OrderCreated</i> para roteamento e filtragem por consumidores.
OrderId (int)	Referência à encomenda criada no nopCommerce.
CustomerId (int)	Identificação do cliente para sistemas downstream.
CreatedAt	Timestamp UTC da criação da encomenda.
Total	Valor total da encomenda.

8.4 Configuração RabbitMQ

A configuração do RabbitMQ foi definida via `definitions.json`, versionado no repositório, garantindo infraestrutura reproduzível:

- **Exchange principal:** *nopcommerce.order.events*, do tipo *topic*, responsável por receber eventos de encomenda publicados pelo nopCommerce.
- **Fila principal:** *nopcommerce.order.created*, responsável por armazenar eventos *OrderCreated* até serem consumidos pelo Order Integration Service.
- **Routing key:** *order.created*, usada para encaminhar eventos de criação de encomenda.
- **Dead Letter Exchange:** *nopcommerce.order.events.dlx*, responsável por receber mensagens rejeitadas ou não processadas corretamente.
- **Dead Letter Queue:** *nopcommerce.order.created.dlq*, usada para armazenar mensagens que falharam no processamento.
- **Credenciais e host:** configurados por variáveis de ambiente no *docker-compose*, evitando valores hardcoded no código.
- **Healthcheck:** configurado para garantir que os serviços dependentes apenas arrancam após o RabbitMQ estar pronto.

8.5 Order Integration Service

O Order Integration Service foi implementado como um serviço independente, executado num container próprio e separado do nopCommerce. Este serviço representa o Order Integration Context definido na arquitetura alvo.

As suas principais responsabilidades são:

- consumir eventos *OrderCreated* da fila *nopcommerce.order.created*;
- coordenar o processamento externo da encomenda;
- invocar o mock de stock;
- invocar o mock de shipping;
- manter o estado de integração da encomenda;
- enviar ACK ao RabbitMQ após processamento com sucesso;
- enviar NACK para encaminhamento para a DLQ em caso de erro;
- expor endpoints HTTP para consulta do estado de integração.

Esta separação reduz o acoplamento do nopCommerce aos sistemas externos e permite adicionar ou alterar integrações sem modificar diretamente o núcleo da plataforma.

8.6 MockStockService — simulação do WMS

O *MockStockService* representa o Warehouse Management System. Nesta implementação, não existe um WMS real; foi criado um mock para simular a reserva de stock.

Este mock permite demonstrar que, após o checkout, a encomenda é processada por um sistema externo ao nopCommerce. Quando a reserva é concluída, o estado de stock da encomenda passa para *Reserved*.

Esta decisão permite preservar a pressão arquitetural do cenário omnichannel sem exigir a implementação completa de um sistema de armazém real.

8.7 MockShippingService — simulação do Shipping Service

O *MockShippingService* representa o sistema externo de shipping. Este serviço simula a criação de um envio e gera um tracking code associado à encomenda.

Após o processamento, a encomenda passa a ter:

- estado de shipping *Shipment Created*;
- tracking code no formato *TRK-{OrderId}-{Code}*;
- estado de integração atualizado no Order Integration Service.

Desta forma, é possível demonstrar que a encomenda segue para um fluxo externo de fulfillment e entrega após ser criada no nopCommerce.

8.8 Integration Status UI

Para demonstrar a visibilidade do estado externo da encomenda, foi criada uma página HTTP exposta pelo Order Integration Service.

Esta página apresenta, por encomenda:

- Order ID;
- Customer ID;
- Order Total;
- Integration Status;
- Stock Status;
- Shipping Status;
- Tracking Code;
- Order Created At;
- Last Updated.

Esta interface permite demonstrar a use case de visibilidade pós-checkout: depois de a encomenda ser criada no nopCommerce, o estado externo produzido pelos sistemas de stock e shipping fica disponível através do Order Integration Service.

Na implementação atual, este estado é mantido em memória. Esta opção é suficiente para a demonstração, mas deve ser considerada uma limitação. Numa versão de produção, este estado deveria ser persistido numa base de dados própria do Order Integration Service.

8.9 Cenário de operação degradada e respetiva recuperação

A implementação permite demonstrar um cenário de degradação controlado:

1. O Order Integration Service é parado.
2. O utilizador realiza um checkout no nopCommerce.
3. O evento *OrderCreated* é publicado no RabbitMQ.
4. Como o consumidor está indisponível, a mensagem permanece na fila *nopcommerce.order.created* com estado *Ready: 1*.
5. O Order Integration Service é iniciado novamente.
6. A mensagem é consumida.
7. O stock é reservado pelo mock WMS.
8. O envio é criado pelo mock Shipping.
9. O Order Integration Service envia ACK ao RabbitMQ.
10. A fila volta a *Ready: 0*.
11. A Integration Status UI apresenta a encomenda como *Completed*.

Este cenário demonstra a resiliência da arquitetura assíncrona: o checkout não depende diretamente do consumidor externo, e a mensagem permanece preservada no broker até poder ser processada.

9. ADRs — Architectural Decision Records

De forma a sustentar a arquitetura proposta, foram identificadas as seguintes decisões arquiteturais, documentadas através de ADRs. As ADRs relativas à infraestrutura assíncrona, ao padrão Outbox, ao contrato de evento e ao Order Integration Service estão consolidadas na implementação atual; as decisões restantes documentam opções de desenho e validação.

ADR 1 — Adoção de comunicação assíncrona entre o sistema central e os sistemas externos	
Contexto	O nopCommerce necessita de interagir com sistemas externos como WMS e Shipping Service. Estas integrações podem estar sujeitas a atrasos, indisponibilidade temporária ou falhas, tornando inadequada uma dependência forte de chamadas síncronas em fluxos críticos.
Decisão	Introdução de comunicação assíncrona entre o nopCommerce e os sistemas externos, recorrendo a um Message Broker para troca de eventos relacionados com encomendas e respetivo processamento. A publicação passou a ser mediada por um padrão Outbox, com relay assíncrono e Publisher Confirms na entrega ao broker.
Alternativas rejeitadas	- Integração síncrona direta entre o nopCommerce e os sistemas externos — bloqueante, quebra continuidade de operação. - Processamento interno no próprio nopCommerce de toda a lógica de comunicação externa — aumenta acoplamento do núcleo.
Justificação	A comunicação assíncrona permite reduzir o acoplamento entre componentes, melhorar a resiliência perante falhas temporárias e suportar consistência eventual entre contextos distintos. O sistema central permanece operacional mesmo quando dependências externas se encontram degradadas.

ADR 2 — Introdução de um Order Integration Service dedicado	
Contexto	A evolução para um contexto omnichannel exige a coordenação de fluxos entre o nopCommerce e múltiplos sistemas externos, com necessidade de gerir retries, correlação de mensagens, tratamento de falhas e lógica de integração específica.
Decisão	Introdução de um Order Integration Service como componente dedicado à integração entre o nopCommerce e os sistemas externos, consumindo eventos do message broker.
Alternativas rejeitadas	- Manter toda a lógica de integração dentro do nopCommerce — aumenta acoplamento e responsabilidade do núcleo. - Integrar diretamente cada sistema externo com o nopCommerce sem componente intermédio — escala mal com múltiplos sistemas.
Justificação	A criação deste serviço permite separar responsabilidades, reduzir o acoplamento do sistema central e concentrar num único contexto as preocupações específicas da integração.

ADR 3 — Separação de ownership de dados entre contextos

Contexto	A arquitetura proposta envolve múltiplos contextos com responsabilidades distintas. A partilha direta da base de dados entre estes contextos criaria dependências fortes e dificultaria a sua evolução independente.
Decisão	Definição de ownership claro dos dados por contexto, evitando a partilha direta da base de dados entre o nopCommerce, o Order Integration Service e os sistemas externos. Comunicação via eventos assíncronos.
Alternativas rejeitadas	- Utilização de uma base de dados partilhada entre múltiplos componentes — acoplamento forte, explicitamente desencorajado no enunciado. - Acesso direto aos dados de outros contextos — viola fronteiras de bounded contexts.
Justificação	A separação de ownership de dados promove desacoplamento, coerência entre responsabilidades e maior independência evolutiva dos componentes.

ADR 4 — Adoção do RabbitMQ como Message Broker

Contexto	Foi necessário seleccionar uma tecnologia concreta de message broker que suportasse os requisitos de comunicação assíncrona, fiabilidade de entrega, DLQ e observabilidade, sem overhead operacional excessivo para este contexto.
Decisão	Adoção do RabbitMQ como message broker, configurado com exchange de tópicos, filas dedicadas, DLQ e Publisher Confirms. Integrado via docker-compose com configuração versionada em definitions.json.
Alternativas rejeitadas	- Apache Kafka — overhead operacional elevado para este contexto; mais adequado a event sourcing de alta escala. - Azure Service Bus / AWS SQS — cloud-native, introduzem dependência de cloud provider e reduzem portabilidade. - Integração direta via HTTP — síncrona, sem suporte nativo a retry ou DLQ.
Justificação	O RabbitMQ oferece o equilíbrio adequado entre funcionalidade e complexidade operacional para este cenário. Publisher Confirms garantem entrega ao broker; a DLQ captura mensagens não processadas sem perda. A interface de gestão facilita a observabilidade imediata.

ADR 5 — Utilização de mocks para WMS e Shipping Service

Contexto	Este cenário pressupõe a integração com os sistemas externos como WMS e Shipping Service. No entanto, implementar sistemas reais de gestão de armazém e transporte excederia o scope do projeto e desviaria o foco da evolução arquitetural pretendida.
Decisão	Representar os sistemas externos através de mocks integrados no Order Integration Service: • <i>MockStockService</i>, que simula a reserva de stock num WMS. • <i>MockShippingService</i>, que simula a criação de envio e geração de tracking code.
Alternativas rejeitadas	• Implementar um WMS real, pois iria aumentar significativamente a complexidade e o tempo de desenvolvimento. • Implementar um Shipping Service real, pelas mesmas razões.
Justificação	Os mocks preservam a pressão arquitetural essencial do cenário: processamento externo ao nopCommerce, comunicação assíncrona, falhas potenciais, recuperação e visibilidade do estado externo. Esta abordagem permite validar a arquitetura sem transformar o projeto na implementação completa de um ecossistema ERP/WMS/Shipping real.

ADR 6 — Exposição do estado de integração pelo Order Integration Service

Contexto	Este cenário exige visibilidade sobre o ciclo da encomenda, incluindo etapas que ocorrem fora do nopCommerce, como reserva de stock e criação de envio.
Decisão	Expor o estado de integração através de endpoints HTTP e de uma página própria no Order Integration Service. Esta página apresenta, por encomenda, o estado de integração, estado de stock, estado de shipping, tracking code, total da encomenda e timestamps.
Alternativas rejeitadas	• Atualizar diretamente a UI original do nopCommerce nesta fase, por aumentar o risco e exigir maior acoplamento ao modelo interno da plataforma. • Mostrar apenas logs no terminal, por ser menos adequado para demonstrar visibilidade operacional. • Persistir imediatamente o estado numa base de dados própria, por aumentar o scope da implementação.
Justificação	A Integration Status UI permite demonstrar a use case de visibilidade pós-checkout sem alterar profundamente o nopCommerce. O estado externo fica visível e consultável, demonstrando consistência eventual e observabilidade do fluxo de integração. Na implementação atual, este estado é mantido em memória, sendo a persistência numa base de dados própria uma evolução futura.

As decisões apresentadas estabelecem a base estrutural da arquitetura alvo, garantindo coerência com o cenário selecionado, com os atributos de qualidade definidos e com os objetivos de evolução da plataforma.

10. Plano de risco e estratégias de mitigação

A arquitetura proposta introduz novos componentes e mecanismos que, embora permitam responder aos desafios deste cenário, também levantam um conjunto de riscos arquiteturais. Foram identificados riscos fechados (mitigados pela implementação atual) e riscos em aberto (dependentes das fases seguintes).

10.1 Riscos arquiteturais

Risco	Prob.	Impacto	Estratégia de Mitigação
Complexidade da comunicação assíncrona	Médio	Médio	Fluxo simplificado (apenas OrderCreated). DLQ para captura de falhas. Configuração versionada via definitions.json. [Fechado para fase atual]
Inconsistência temporária dos dados	Alto	Médio	Arquitetura e contrato de evento definidos. Demonstração de estados intermédios depende do mock externo. [Em aberto]
Fiabilidade das integrações externas (WMS, Shipping)	Alto	Alto	Publisher Confirms e DLQ implementados. Retry do lado do consumidor e recuperação completa dependem do mock externo. [Em aberto]
Observabilidade insuficiente em sistemas distribuídos	Médio	Alto	Logs no publisher e checkout implementados. RabbitMQ Management UI disponível. Tracing distribuído entre serviços planeado para fase seguinte. [Parcialmente fechado]
Broker como ponto único de falha	Baixo	Alto	Checkout tolerante a falha do broker (try/catch implementado). Filas durable. Clustering RabbitMQ planeado se necessário. [Fechado para continuidade de operação]
Evento perdido entre checkout e broker	Médio	Alto	Padrão Outbox implementado na presente arquitetura; a persistência transacional elimina a janela de perda entre a criação da encomenda e a publicação no broker. [Fechado]
Falta de consumidor externo funcional	Alto	Médio	Risco em aberto: o consumer existe (OrderCreatedEventConsumer) mas não está ligado a um sistema externo real ou stub. Bloqueia a validação de QA1, QA2 e QA3 na sua totalidade.

10.2 Estratégias de mitigação adotadas

As seguintes estratégias foram efetivamente aplicadas na implementação final:

- **Comunicação assíncrona via RabbitMQ:** reduz o acoplamento entre o nopCommerce e os sistemas externos.
- **Order Integration Service:** concentra a lógica de integração num serviço independente, evitando que o nopCommerce conheça diretamente WMS ou Shipping.
- **ACK/NACK:** garante que a mensagem só é removida da fila após processamento bem-sucedido.
- **Dead Letter Queue:** permite isolar mensagens que falham no processamento.
- **Mocks de WMS e Shipping:** permitem validar o fluxo omnichannel sem implementar sistemas externos completos.
- **Integration Status UI:** permite observar o estado externo da encomenda após checkout.
- **Healthcheck do RabbitMQ:** reduz problemas de arranque em que o OIS ou o nopCommerce tentavam comunicar com o broker antes de este estar pronto.

10.3 Estratégias de mitigação futuras

Apesar da implementação realizada, permanecem algumas melhorias futuras relevantes:

- **Retries com backoff:** ajustar políticas de reprocessamento para evitar picos de retries quando o broker ou o consumidor apresentam degradação temporária.
- **Persistência do estado do OIS:** substituir o estado em memória por uma base de dados própria do Order Integration Service.
- **Atualização direta do estado no nopCommerce:** propagar o estado final do processamento externo para a experiência original do utilizador.

10.4 Conclusão

A identificação destes riscos e a definição de estratégias de mitigação permitem reduzir a incerteza associada à arquitetura proposta, garantindo que as principais decisões podem ser testadas e ajustadas de forma iterativa. Os riscos fechados demonstram que a infraestrutura assíncrona está operacional; os riscos em aberto definem claramente o trabalho a realizar na fase seguinte.

11. Evolution Roadmap

A evolução da arquitetura proposta é realizada de forma incremental, garantindo a continuidade do funcionamento do sistema e reduzindo o risco associado à introdução de novos componentes. O roadmap define fases concluídas, em curso e planejadas.

11.1 Fases de implementação

Fase	Objetivo	Estado	Evidência / Próximo passo
1	Introdução de observabilidade base	Concluído	Logs no publisher e checkout. RabbitMQ Management UI.
2	Introdução do Message Broker (RabbitMQ)	Concluído	alterações implementadas — exchange, filas, DLQ, docker-compose, definitions.json.
3	Emissão de eventos de encomenda	Concluído	alterações implementadas — <i>OrderCreatedEvent</i> , <i>OrderProcessingService</i> , Publisher Confirms.
4	Order Integration Service — consumidor	Concluído	<i>OrderCreatedEventConsumer</i> criado. Integração com WMS/Shipping pendente.
5	Integração com WMS mock	Concluído	<i>MockStockService</i> implementado para simular reserva de stock após receção do evento.
6	Integração com Shipping mock	Concluído	<i>MockShippingService</i> implementado para simular criação de envio e geração de tracking code.
7	ACK/NACK e DLQ	Concluído	O Order Integration Service envia ACK após processamento com sucesso e NACK em caso de falha, permitindo encaminhamento para DLQ.
8	Integration Status UI	Concluído	Página para consultar o estado externo das encomendas.
9	Demonstração de operação degradada e recuperação	Concluído	Com o OIS parado, a mensagem permanece na queue; após reinício, é consumida e processada.
10	Padrão Outbox	Concluído	Persistência transacional do evento antes da publicação no broker, eliminando o risco de evento perdido entre checkout e RabbitMQ.
11	Persistência do estado do OIS	Concluído	Substituição do estado em memória por uma base de dados própria do Order Integration Service.
12	Atualização direta do estado no nopCommerce	Futuro	Propagar o estado final de stock/shipping para a experiência original do utilizador no nopCommerce.

11.2 Ordem de evolução

Conforme descrito, toda a evolução seguiu uma abordagem incremental. Numa primeira fase, foi introduzida a infraestrutura base de comunicação assíncrona através do RabbitMQ. De seguida, o nopCommerce passou a emitir eventos relativos à criação de uma encomenda.

Numa fase posterior, foi introduzido o Order Integration Service como consumidor independente desses eventos. Este serviço passou a coordenar o processamento externo da encomenda, invocando o *MockStockService* e o *MockShippingService*. Por fim, foi adicionada a Integration Status UI, permitindo demonstrar a visibilidade do estado externo da encomenda após o checkout.

Esta ordem permitiu validar cada componente de forma isolada e reduzir o risco da implementação: primeiro a publicação de eventos, depois o broker, depois o consumidor, depois os sistemas externos simulados e, finalmente, a visibilidade do estado.

11.3 Coexistências durante a transição

Durante o processo de evolução, o nopCommerce continuou a operar como sistema principal de experiência de compra, catálogo, carrinho, checkout e criação de encomendas. Os novos componentes foram introduzidos de forma incremental, sem substituir o fluxo principal da aplicação.

O checkout manteve-se funcional mesmo com o processamento externo desacoplado. O RabbitMQ passou a atuar como ponto intermédio entre o nopCommerce e o Order Integration Service, permitindo que o processamento externo ocorra de forma assíncrona.

Na implementação atual, o WMS e o Shipping Service são representados por mocks. Esta opção permite demonstrar o comportamento arquitetural pretendido sem exigir a implementação completa de sistemas externos reais. A coexistência entre o nopCommerce e os novos componentes é garantida através de eventos e não por partilha direta da base de dados.

11.4 Conclusão

A definição deste roadmap permite estruturar a evolução da arquitetura de forma progressiva e controlada, assegurando que cada etapa contribui para a validação da solução e para a redução dos riscos associados à mudança.

12. Feasibility Spike

Tendo em conta a análise realizada às mudanças a efetuar do ponto de vista da arquitetura atual da plataforma nopCommerce e o cenário escolhido, foi estruturado e executado um feasibility spike com o objetivo de clarificar a parte mais arriscada das alterações propostas: a integração assíncrona e respetiva comunicação entre o sistema central e componentes externos.

12.1 Objetivo

Validar a viabilidade da utilização de comunicação assíncrona, baseada em eventos, para integração entre o nopCommerce e sistemas externos, garantindo suporte a resiliência, desacoplamento e consistência eventual.

Mais concretamente, o spike procurou validar:

- se o nopCommerce consegue emitir eventos após checkout;
- se o RabbitMQ consegue armazenar e encaminhar esses eventos;
- se um serviço externo independente consegue consumir os eventos;
- se o processamento externo pode ocorrer sem bloquear a experiência do utilizador;
- se o estado externo da encomenda pode ficar visível após processamento assíncrono;
- se a arquitetura consegue recuperar quando o consumidor externo está temporariamente indisponível.

12.2 Descrição e resultados — fluxo assíncrono completo

O spike foi implementado e permitiu validar os seguintes aspetos:

- **Emissão de eventos:** após um checkout bem-sucedido, o nopCommerce cria um *OrderCreatedEvent* com os dados essenciais da encomenda.
- **Publicação no RabbitMQ:** o evento é publicado no exchange `nopcommerce.order.events` com a routing key `order.created`.
- **Armazenamento em fila:** o *RabbitMQ* encaminha a mensagem para a fila `nopcommerce.order.created`.
- **Consumo externo:** o *Order Integration Service* consome eventos da fila de forma independente do nopCommerce.
- **Processamento por WMS mock:** o *MockStockService* simula a reserva de stock.
- **Processamento por Shipping mock:** o *MockShippingService* simula a criação de envio e gera um tracking code.
- **Acknowledgement:** após processamento bem-sucedido, o *Order Integration Service* envia *ACK* ao *RabbitMQ*, removendo a mensagem da fila.
- **Visibilidade do estado:** a *Integration Status UI* apresenta o estado final da encomenda, incluindo stock reservado, envio criado, tracking code e timestamps.
- **Configuração reproduzível:** a infraestrutura é definida através de `docker-compose`, `definitions.json` e `rabbitmq.conf`.

12.3 Runtime challenge demonstrável — Order Integration Service indisponível

O cenário de degradação principal demonstrado foi a indisponibilidade temporária do Order Integration Service.

Cenário: o Order Integration Service encontra-se parado durante o checkout.

Comportamento observado:

- O utilizador realiza o checkout no nopCommerce.
- A encomenda é criada no nopCommerce.
- O evento *OrderCreated* é publicado no RabbitMQ.
- Como o consumidor está indisponível, a mensagem permanece na fila *nopcommerce.order.created*.
- No RabbitMQ Management UI, a fila apresenta a mensagem como *Ready: 1*.

Este comportamento demonstra que a indisponibilidade do consumidor externo não implica perda imediata da mensagem e não obriga o nopCommerce a conhecer ou contactar diretamente os sistemas externos.

12.4 Conclusão

O feasibility spike valida a decisão arquitetural mais crítica da proposta — a integração assíncrona baseada em eventos com o RabbitMQ — reduzindo o risco associado e garantindo maior confiança na implementação da arquitetura alvo.

13. Runtime challenge e demonstração

Neste capítulo abordamos os cenários de runtime demonstráveis na implementação final. A demonstração foi estruturada para mostrar, não apenas o funcionamento normal da arquitetura, mas também o comportamento do sistema perante degradação e recuperação, conforme exigido pelo cenário Omnichannel Commerce Core.

13.1 Operação normal — checkout com processamento externo assíncrono

No cenário de operação normal, o utilizador realiza uma encomenda no nopCommerce e o processamento externo ocorre de forma assíncrona através do RabbitMQ e do Order Integration Service.

O fluxo demonstrado é o seguinte:

1. O utilizador completa o checkout no nopCommerce.
2. A encomenda é criada e persistida no nopCommerce.
3. É criado um evento *OrderCreated*.
4. O evento é guardado no Outbox do nopCommerce na mesma transação da encomenda.
5. Um relay assíncrono lê o evento pendente do *Outbox*.
6. O relay publica o evento no *RabbitMQ*.
7. A mensagem é encaminhada para a fila *nopcommerce.order.created*.
8. O Order Integration Service consome a mensagem.
9. O *MockStockService* simula a reserva de stock.
10. O *MockShippingService* simula a criação do envio e gera um tracking code.
11. O Order Integration Service atualiza o estado da integração na sua base de dados SQLite.
12. O *Order Integration Service* envia ACK ao RabbitMQ.
13. A mensagem é removida da fila.
14. A Integration Status UI apresenta a encomenda com estado Completed, stock Reserved, shipping Shipment Created e tracking code associado.

Este cenário demonstra o fluxo principal da arquitetura implementada e valida a integração assíncrona entre o nopCommerce e sistemas externos simulados.

13.2 Operação com o Order Integration Service indisponível

O principal runtime challenge demonstrado consiste na indisponibilidade temporária do consumidor externo, ou seja, do Order Integration Service.

Neste cenário:

1. O Order Integration Service é parado.
2. O utilizador realiza um novo checkout no nopCommerce.
3. A encomenda é criada no nopCommerce.
4. O evento *OrderCreated* é guardado no Outbox.
5. O relay assíncrono publica o evento no RabbitMQ.
6. Como o consumidor está indisponível, a mensagem permanece na fila *nopcommerce.order.created*.
7. No RabbitMQ Management UI, a fila apresenta a mensagem como *Ready: 1*.

Este comportamento demonstra que o nopCommerce não está diretamente acoplado ao processamento externo. Mesmo com o Order Integration Service indisponível, a mensagem não é perdida e fica preservada no broker até existir um consumidor disponível.

13.3 Recuperação — consumidor volta a estar disponível

Após o cenário de degradação, o Order Integration Service é iniciado novamente.

O comportamento observado é o seguinte:

1. O Order Integration Service volta a ligar-se ao RabbitMQ.
2. A mensagem pendente é consumida da fila.
3. O *MockStockService* reserva o stock da encomenda.
4. O *MockShippingService* cria o envio e gera o tracking code.
5. O estado de integração é atualizado no Order Integration Service.
6. O serviço envia ACK ao RabbitMQ.
7. A fila volta a apresentar *Ready: 0*.
8. A Integration Status UI apresenta a encomenda como processada com sucesso.

Este cenário demonstra a capacidade de recuperação da arquitetura: uma falha temporária no consumidor externo não provoca perda da mensagem nem impede o processamento posterior da encomenda.

13.4 Visibilidade do estado externo da encomenda

Para demonstrar a visibilidade do ciclo de encomendas após o checkout, foi criada uma página que refere o estado no Order Integration Service.

Esta página apresenta:

- Order ID;
- Customer ID;
- Order Total;
- Integration Status;
- Stock Status;
- Shipping Status;
- Tracking Code;
- Order Created At;
- Last Updated.

Esta interface permite observar o resultado do processamento externo da encomenda, tornando visível que o stock foi reservado e que o envio foi criado por sistemas externos simulados.

14. Evidence Pack

O evidence pack reúne as evidências disponíveis nesta fase. As evidências de implementação estão fechadas; as evidências de demonstração em runtime ficam em aberto para complementar na fase seguinte.

14.1 Evidência de implementação

Evidência	Evidência
Repositório	Branch principal do projeto, partindo de uma versão base do nopCommerce previamente analisada.
nopCommerce	Alterações no fluxo de criação de encomenda para emissão de eventos <i>OrderCreated</i> .
Evento de integração	Alterações no fluxo de criação de encomenda para emissão de eventos <i>OrderCreated</i> .
RabbitMQ	Configuração de exchange, fila principal, DLQ, bindings, vhost, credenciais e healthcheck via <i>docker-compose</i> , <i>definitions.json</i> e <i>rabbitmq.conf</i> .
Order Integration Service	Serviço independente criado em <i>src/ExternalServices/OrderIntegrationService</i> , executado em container próprio.
WMS mock	Implementação do <i>MockStockService</i> , responsável por simular a reserva de stock.
Shipping mock	Implementação do <i>MockShippingService</i> , responsável por simular a criação de envio e tracking code.
Estado de integração	Implementação do <i>OrderIntegrationStateStore</i> , responsável por manter o estado externo das encomendas processadas.
Integration Status UI	Página HTTP exposta pelo OIS para consultar o estado de integração das encomendas.
Fiabilidade	Utilização de ACK/NACK no consumidor e DLQ para mensagens não processadas corretamente.

15. Rastreabilidade — drivers, QA, ADRs e implementação

A tabela seguinte estabelece a rastreabilidade entre os drivers de negócio identificados, os quality attribute scenarios, as decisões arquiteturais (ADRs) e a implementação realizada, indicando o estado de cada linha.

Driver de Negócio	Quality Attribute	ADR	Implementação	Estado
Integração com sistemas externos	QA1 — Resiliência	ADR 1, ADR 4	RabbitMQ, Outbox, <i>OrderCreatedEventPublisher</i>	Parcialmente implementado
Continuidade de operação	QA5 — Continuidade	ADR 1	try/catch no checkout () e persistência transacional do evento	Implementado
Experiência unificada	QA2 — Consistência eventual	ADR 3	Data ownership separado; eventos assíncronos	A validar
Capacidade de evolução	QA3 — Desacoplamento	ADR 2	Order Integration Service, consumer separado	Parcialmente implementado
Visibilidade do ciclo	QA4 — Observabilidade	ADR 4	Logs publisher; RabbitMQ Management; DLQ	Parcialmente implementado
Fiabilidade de mensagens	QA1 — Resiliência	ADR 4	Outbox, Publisher Confirms, DLQ	Implementado
Separação de dados	Todos os QA	ADR 3	Sem base de dados partilhada; eventos como contrato	Implementado

16. Conclusão

O trabalho desenvolvido permitiu evoluir a arquitetura do nopCommerce de acordo com o Cenário C — Omnichannel Commerce Core. A solução implementada introduz comunicação assíncrona com RabbitMQ, um padrão outbox, um Order Integration Service independente e dois sistemas externos simulados: WMS e Shipping.

Com esta evolução, o nopCommerce continua responsável pelo checkout e pela criação da encomenda, enquanto o processamento externo passa a ser feito de forma desacoplada. Após uma encomenda ser criada, é publicado um evento *OrderCreated*, consumido pelo Order Integration Service, que simula a reserva de stock, a criação do envio e a geração de um tracking code.

A implementação permite demonstrar o fluxo normal e também um cenário de degradação: quando o Order Integration Service está parado, a mensagem permanece na fila do RabbitMQ; quando o serviço volta a estar disponível, a mensagem é consumida e processada. Quando a falha ocorre antes da publicação no broker, o outbox preserva o evento até ao reprocessamento. A página de Integration Status permite visualizar o estado final da encomenda após o processamento externo.

Apesar de existirem limitações, como o uso de mocks e o estado em memória no OIS, a solução valida os principais objetivos arquiteturais definidos: desacoplamento, resiliência, continuidade de operação, observabilidade e consistência eventual.